

系统分析师易混淆知识点

易混淆点 1: 图论应用	2
易混淆点 2: 索引文件结构	2
易混淆点 3: 企业应用集成	3
易混淆点 4: 电子政务	4
易混淆点 5: 投资回收期	4
易混淆点 6: 需求获取	5
易混淆点 7: UML 图	6
易混淆点 8: UML 图关系	8
易混淆点 9: 设计模式	8
易混淆点 10: 软件维护	10
易混淆点 11: 安全协议	10
易混淆点 12: 五大架构风格	11
易混淆点 13: 关系代数	12
易混淆点 14: 规范化理论	14
易混淆点 15: TCP/IP 协议族	15
易混淆点 16: CMM 和 CMMI	16
易混淆点 17: 逆向工程	17

易混淆点 1：图论应用

1.最小生成树

在连通的带权图的所有生成树中，权值和最小的那棵生成树（包含图中所有顶点的树），称作最小生成树。求带权连通无向图的最小生成树的算法有普里姆（最近顶点）算法和克鲁斯卡尔（最短边）算法。

解题关键：每次找最短边，直到将所有节点连成一棵没有回路的树。

2.最短路径

带权图的最短路径问题即求两个顶点间长度最短的路径。其中路径长度不是指路径上边数的总和，而是指路径上各边的权值总和。路径长度的具体含义取决于边上权值所代表的意义。算法有迪杰斯特拉算法。

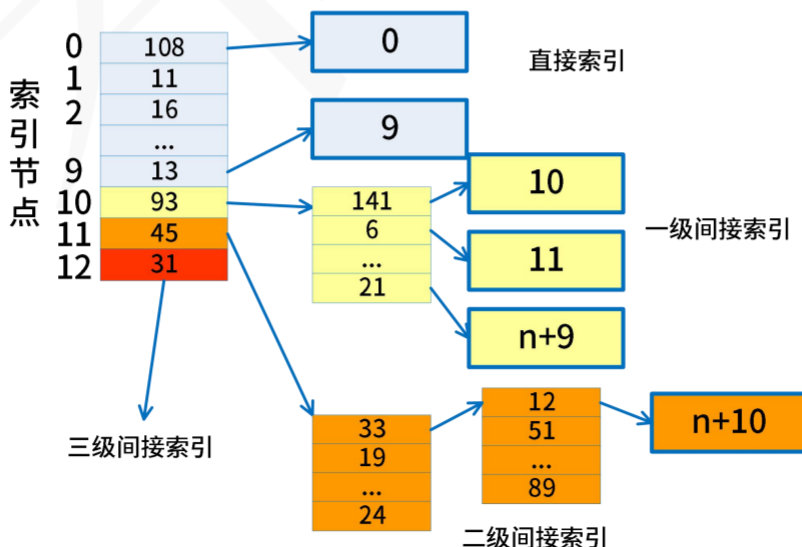
解题关键：从源点出发，计算源点到下一节点的最短路径。后续分析时，每次只考虑当前节点前一个节点的最短路径长度，直到终点位置，分析其最短路径。

3.网络与最大流量

许多应用包含了流量问题。例如，公路系统中有车辆流，控制系统中有信息流，网络系统中有数据流，金融系统中有现金流等。在实际应用中，很多时候需要寻求最大流量问题的解。最大流量问题是一个特殊的线性规划问题。

解题关键：每次查找并抽取起点到终点的最大流量路径，直到没有可达路径为止，抽取的流量总和即为最大流量。

易混淆点 2：索引文件结构



(1) 索引节点对应的索引方式一般题干会给出, 没有给出的默认按照如图所示方式理解, 下面的文件大小依图给出计算过程。

(2) 根据物理块大小 (假设 1KB) 和地址项长度 (假设 4B), 可以计算存放间接索引的物理块可以存放的地址项个数: 物理块大小/地址项长度, 向下取整 ($1\text{KB}/4\text{B}=256$, 注意单位和进制转换)。

(3) 直接索引 (即索引节点直接指向实际存储文件的物理块), 能够表示的逻辑页号范围是 0-9, 能够表示的文件大小是 $10 \times 1\text{KB}$ 。【访问 1 次对应磁盘找到数据】

(4) 一级间接索引 (即索引节点指向的物理块存放的是一级间接索引表的地址项, 共 256 个, 可以指向 256 个实际存储文件的物理块), 能够表示的逻辑页号范围是 10-265, 能够表示的文件大小是 $256 \times 1\text{KB}$ 。【访问 2 次对应磁盘找到数据】

(5) 二级间接索引 (即索引节点指向的物理块存放的是二级间接索引表的地址项, 共 256 个, 可以指向 256 个一级间接索引表地址项的物理块, 每个物理块指向实际存储文件的地址项有 256 个, 最终指向的物理块共有 256×256 个), 能够表示的逻辑页号范围是 266-65801, 能够表示的文件大小是 65536KB。【访问 3 次对应磁盘找到数据】。

易混淆点 3: 企业应用集成

不同维度的集成划分:

(1) 按集成点划分:

	集成点	效果	解题关键点
界面集成	界面层	统一入口, 产生“整体”感觉	“整体”感觉 最小代价实现一体化操作
数据集成	数据层	不同来源的数据逻辑或物理上“集中”	其他集成方法的基础
控制集成	应用逻辑层	调用其它系统已有方法, 达到集成效果	/
业务流程集成 (过程集成)	应用逻辑层	跨企业, 或优化流程而非直接调用	企业之间的信息共享能力
门户集成	/	将内部系统对接到互联网上	发布到互联网上

(2) 按传输方式划分:

	特点
消息集成	数据量小, 交互频繁, 立即地, 异步
共享数据库	交互频繁, 立即地, 同步
文件传输	数据量大, 交互频度小, 即时性要求低 (月末, 年末)

易混淆点 4: 电子政务

电子政务主要有 3 类角色: 政府 (Government)、企 (事) 业单位 (Business) 及公民 (Citizen)。如果有第 4 类就是公务员 (Employee)。

类型	应用
G2G	基础信息的采集、处理和利用, 如: 人口信息、地理信息 各级政府决策支持
G2E	政府内部管理系统
G2B	政府给企业单位颁发【各种营业执照、许可证、合格证、质量认证】
B2G	企业向政府缴税 企业向政府供应各种商品和服务【含竞/投标】 企业向政府提建议, 申诉
G2C	社区公安和水、火、天灾等与公共安全有关的信息 户口、各种证件和牌照的管理
C2G	个人应向政府缴纳的各种税款和费用 个人向政府反馈民意【征求群众意见】 报警服务【盗贼、医疗、急救、火警等】

易混淆点 5: 投资回收期

1、静态投资回收期 (不考虑货币的时间价值因素)

如果投资在建设期 m 年内分期投入, t 年的投资为 P_t , t 年的净现金收入为 $(CI - CO)_t$, 则能够使下面公式成立的 T 即为静态投资回收期。

$$\sum_{t=0}^m P_t = \sum_{t=0}^T (CI - CO)_t$$

2、动态投资回收期 (考虑资金的时间价值因素)

如果考虑资金的时间价值, 则动态投资回收期 I 的计算公式, 应满足

$$\sum_{t=0}^{T_p} \frac{(CI - CO)_t}{(1 + i)^t} = 0$$

计算动态投资回收期的实用公式为: $T_p = \text{累计折现值开始出现正值的年份数} - 1 + |\text{上年累计折现值}| / \text{当年折现值}$

3、投资回收期=1/投资回收期*100%

4、投资回报率=投资收益率=投资收益/投资总额*100%

5、年均投资收益率=运营期年均净收益/投资总额*100%

6、现值指数=投资净收益/投资成本*100%=净现值率+100%

注：做题时需要仔细审题，题目要求的是动态投资回收期还是静态投资回收期

易混淆点 6：需求获取

(1) 需求获取方法介绍

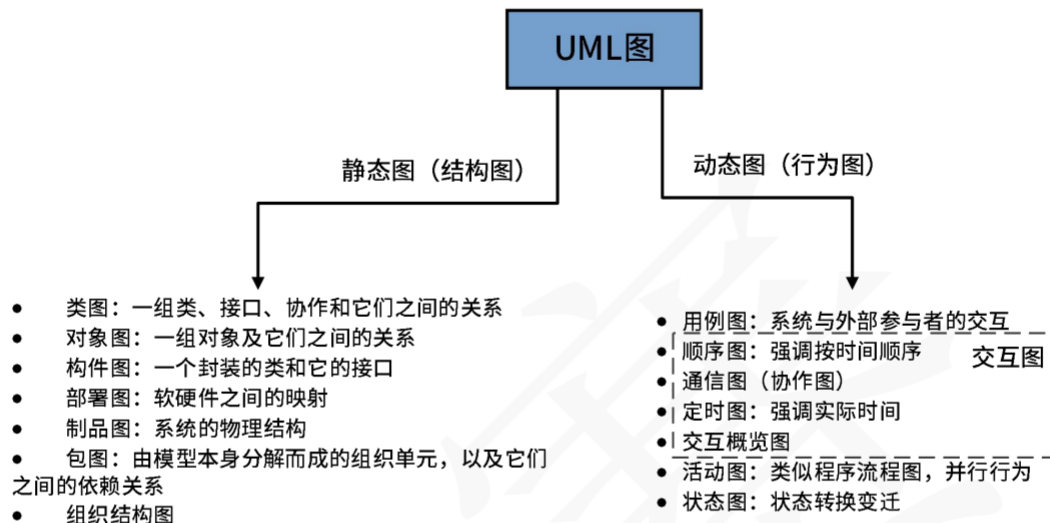
方法	特点
用户访谈	1 对 1-3，有代表性的用户，了解主观想法，交互好。成本高，要有领域知识支撑
问卷调查	用户多，无法一一访谈，成本低
采样 (抽样调查)	基于数理统计，降低成本，快速获取 采样技术包括：随机采样、分层采样、聚类采样等，具体采用哪种方法根据文档数量、样本大小和实际情况决定
情节串联板 【原型前身】	一系列图片，通过这些图片来讲故事。类型包括：被动式、主动式和交互式
联合需求计划	高度组织的群体会议，各方参与，了解想法，消除分歧，交互好，成本高
需求记录技术	需求记录工具有：任务卡片、场景说明、用户故事和 Volere 白卡等

(2) 需求获取方法对比

属性方法	成本	信息深度	信息广度	信息集成	用户参与度	适用性
用户访谈	高	高	低	低	中等	复杂需求
联合需求计划	高	高	中等	高	高	团队协作
问卷调查	低	中等	高	低	低	大规模用户
文档分析	低	低	中等	低	低	遗留系统
实地观察	中等	低	低	低	中等	行为分析

情节串联板【原型】	高	中等	低	中等	高	需求模糊
-----------	---	----	---	----	---	------

易混淆点 7: UML 图



UML 2.0 包括 14 种图，分别如下：

(1) 类图 (class diagram)。类图描述一组类、接口、协作和它们之间的关系。类图给出了系统的静态设计视图，活动类的类图给出了系统的静态进程视图。

(2) 对象图 (object diagram)。对象图描述一组对象及它们之间的关系。对象图描述了在类图中所建立的事物实例的静态快照。对象图从真实案例或原型案例的角度建立的。

(3) 构件图 (component diagram)。构件图描述一个封装的类和它的接口、端口，以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。

(4) 组合结构图 (composite structure diagram)。组合结构图描述结构化类（例如，构件或类）的内部结构，包括结构化类与系统其余部分的交互点。组合结构图用于画出结构化类的内部内容。

(5) 用例图 (use case diagram)。用例图描述一组用例、参与者及它们之间的关系。用例图给出系统的静态用例视图。

(6) 顺序图 (sequence diagram)。顺序图是一种交互图 (interaction diagram)，交互图展现了一种交互，它由一组对象或参与者以及它们之间可能发送的消息构成。交互图专注于系统的动态视图。顺序图是强调消息的时间次序的交互图。

(7) 通信图 (communication diagram)。通信图也是一种交互图，它强调收发消息的对象或参与者的结构组织。

(8) 定时图 (timing diagram)。定时图也是一种交互图，它强调消息跨越不同对象或参与者的实际时间，而不仅仅只是关心消息的相对顺序。

(9) 状态图 (state diagram)。状态图描述一个状态机，它由状态、转移、事件和活动组成。状态图给出了对象的动态视图。它对于接口、类或协作的行为建模尤为重要，而且它强调事件导致的对象行为，这非常有助于对反应式系统建模。

(10) 活动图 (activity diagram)。活动图将进程或其他计算结构展示为计算内部一步步的控制流和数据流。活动图专注于系统的动态视图。它对系统的功能建模和业务流程建模特别重要，并强调对象间的控制流程。

(11) 部署图 (deployment diagram)。部署图描述对运行时的处理节点及在其中生存的构件的配置。部署图给出了架构的静态部署视图，通常一个节点包含一个或多个部署图。

(12) 制品图 (artifact diagram)。制品图描述计算机中一个系统的物理结构。制品包括文件、数据库和类似的物理比特集合。制品图通常与部署图一起使用。制品也给出了它们实现的类和构件。

(13) 包图 (package diagram)。包图描述由模型本身分解而成的组织单元，以及它们之间的依赖关系。

(14) 交互概览图 (interaction overview diagram)。交互概览图是活动图和顺序图的混合物。

希赛专家解读：

1、活动图与流程图关系

活动图描述的是对象活动的顺序关系所遵循的规则，它着重表现系统的行为，而非处理过程；而流程图着重描述处理过程。

流程图一般都限于顺序进程，而活动图则可以支持并发进程。

活动图是面向对象的，而流程图是面向过程的。

2、活动图与状态图对比

状态图主要用于描述一个对象在其生存期间的动态行为，表现一个对象所经历的状态序列，引起状态转移的事件 (event)，以及因状态转移而伴随的动作 (action)。

活动图可以用于描述系统的工作流程和并发行为。活动图其实可看作状态图的特殊形式，活动图中的一个活动结束后将立即进入下一个活动（在状态图中状态的转移可能需要事件的触发）。

两者最大的区别是：状态图侧重于描述行为的结果，而活动图侧重描述行为的动作。其次活动图可描述并发行为，而状态图不能。

易混淆点 8: UML 图关系

(1) 用例关系包括: 包含关系、扩展关系、泛化关系。

1、包含关系: 其中这个提取出来的公共用例称为抽象用例, 而把原始用例称为基本用例或基础用例: 当可以从两个或两个以上的用例中提取公共行为时, 应该使用包含关系来表示它们。

2、扩展关系: 如果一个用例明显地混合了两种或两种以上的不同场景, 即根据情况可能发生多种分支, 则可以将这个用例分为一个基本用例和一个或多个扩展用例, 这样使描述可能更加清晰。

3、泛化关系: 当多个用例共同拥有一种类似的结构和行为的时候, 可以将它们的共性抽象成为父用例, 其他的用例作为泛化关系中的子用例。在用例的泛化关系中, 子用例是父用例的一种特殊形式, 子用例继承了父用例所有的结构、行为和关系。

(2) 类图/对象图关系:

1、依赖关系: 一个事物发生变化影响另一个事物。

2、泛化关系: 特殊/一般关系

3、关联关系: 描述了一组链, 链是对象之间的连接。

4、聚合关系: 整体与部分生命周期不同。

5、组合关系: 整体与部分生命周期相同。

6、实现关系: 接口与类之间的关系

注: 用例关系只有包含、扩展和泛化, 不要和类图的关系弄混淆。

易混淆点 9: 设计模式

(1) 创建型模式

设计模式名称	简要说明	速记关键字
Factory Method 工厂方法模式	定义了创建对象的接口, 它允许子类决定实例化哪个类	动态生产对象
Abstract Factory 抽象工厂模式	提供一个接口, 可以创建一系列相关或相互依赖的对象, 而无需指定它们具体的类	生产成系列对象
Builder 构建器模式	将一个复杂类的表示与其构造相分离, 使得相同的构建过程能够得出不同的表示	复杂对象构造
Prototype 原型模式	允许对象在了解要创建对象的确切类以及如何创建等细节的情况下创建自定义对象。通过拷贝原型对象来创建新的对象	克隆对象
Singleton	确保一个类只有一个实例, 并且提供了对该类的全局访问入口	单实例

单例模式

(2) 结构型模式

设计模式名称	简要说明	速记关键字
Adapter 适配器模式	将一个类的接口转换成用户希望得到的另一种接口。它使原本不相容的接口得以协同工作	转换接口
Bridge 桥接模式	将一个复杂的组件分成两个独立的但又相关的继承层次结构 将类的抽象部分和它的实现部分分离开来, 使它们可以独立地变化	继承树拆分
Composite 组合模式	创建树型层次结构来改变复杂性, 同时允许结构中的每一个元素操作同一个接口。用于表示“整体-部分”的层次结构	树形目录结构
Decorator 装饰模式	在不修改对象外观和功能的情况下添加或者删除对象功能 即动态地给一个对象添加一些额外的职责	动态附加职责
Facade 外观模式	子系统中的一组接口提供了一个统一的接口	对外统一接口
Flyweight 享元模式	可以通过共享对象减少系统中低等级的、详细的对象数目 提供支持大量细粒度对象共享的有效方法	汉字编码
Proxy 代理模式	为控制对初始对象的访问提供了一个代理或者占位符对象	快捷方式

(3) 行为型模式

设计模式名称	简要说明	速记关键字
Chain of Responsibility 职责链模式	可以在系统中建立一个链, 这样消息可以在首先接收到它的级别处被处理, 或者可以定位到可以处理它的对象	传递职责
Command 命令模式	在对象中封装了请求, 这样就可以保存命令, 将该命令传递给方法以及像任何其他对象一样返回该命令	日志记录, 可撤销
Interpreter 解释器模式	可以解释定义其语法表示的语言, 还提供了用表示来解释语言中的语句的解释器	虚拟机的机制
Iterator 迭代器模式	为集合中的有序访问提供了一致的方法, 而该集合是独立于基础集合, 并与之相分离的	数据集
Mediator 中介者模式	通过引入一个能够管理对象间消息分布的对象, 简化了系统中对象间的通信	不直接引用

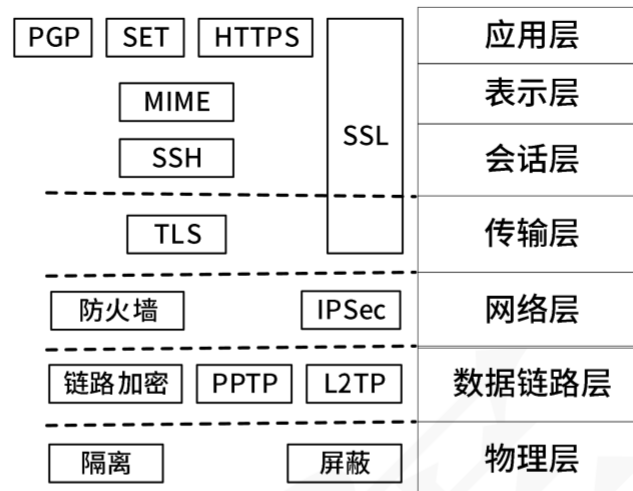
Memento 备忘录模式	保持对象状态的“快照” (snapshot)，这样对象可以在不向外界公开其内容的情况下返回到它的最初状态	游戏存档
Observer 观察者模式	为组件向相关接收方广播消息提供了灵活的方法 定义对象间的一种一对多的依赖关系	订阅、广播、联动
State 状态模式	允许一个对象在其内部状态改变时改变它的行为	状态变成类
Strategy 策略模式	定义一系列算法，把它们一个个封装起来，并且使它们之间可互相替换，从而让算法可以独立于使用它的用户而变化	多方案切换
Template Method 模板方法模式	提供了在不重写方法的前提下允许子类重载部分方法的方法	框架
Visitor 访问者模式	提供了一种方便的、可维护的方法来表示在对象结构元素上要进行的操作	数据与操作分离

易混淆点 10：软件维护

- 1、正确性维护【修 BUG】：识别和纠正软件错误/缺陷，测试不可能发现所有错误。
- 2、适应性维护【应变】：指使应用软件适应环境变化【外部环境、数据环境】而进行的修改。
- 3、完善性维护【新需求】：扩充功能和改善性能而进行的修改。
- 4、预防性维护【针对未来】：为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使用系统适应各类变化而不被淘汰。经典实例：【专用】改【通用】。

注：适应性维护和预防性维护比较像，要注意区分。一个是适应环境变化，一个是适应未来的环境变化

易混淆点 11：安全协议



PGP (Pretty Good Privacy) : PGP 是一个基于 RSA 的邮件加密软件, 可以用它对邮件保密以防止非授权者阅读, 它还能对邮件加上数字签名, 从而使收信人可以确信邮件发送者。

SSL (Secure Sockets Layer) : SSL 是一个传输层的安全协议, 用于在 Internet 上传送机密文件。SSL 协议由握手协议、记录协议和警报协议组成。

TLS (Transport Layer Security) : 用于保护网络通信的传输层安全协议, TLS 是 SSL 的升级版。它们通过加密通信内容和验证通信双方身份来保护网络通信的安全性。

SET (Secure Electronic Transaction) : 安全电子交易协议。电子商务, 身份认证。普遍的说法是将其归为应用层。

IPsec (Internet Protocol Security) : IPsec 是一个工业标准网络安全协议, 为 IP 网络通信提供透明的安全服务, 保护 TCP/IP 通信免遭窃听和篡改, 可以有效抵御网络攻击, 同时保持易用性。

注: SSL 协议工作于应用层至传输层, 但却属于传输层协议哦。

易混淆点 12: 五大架构风格

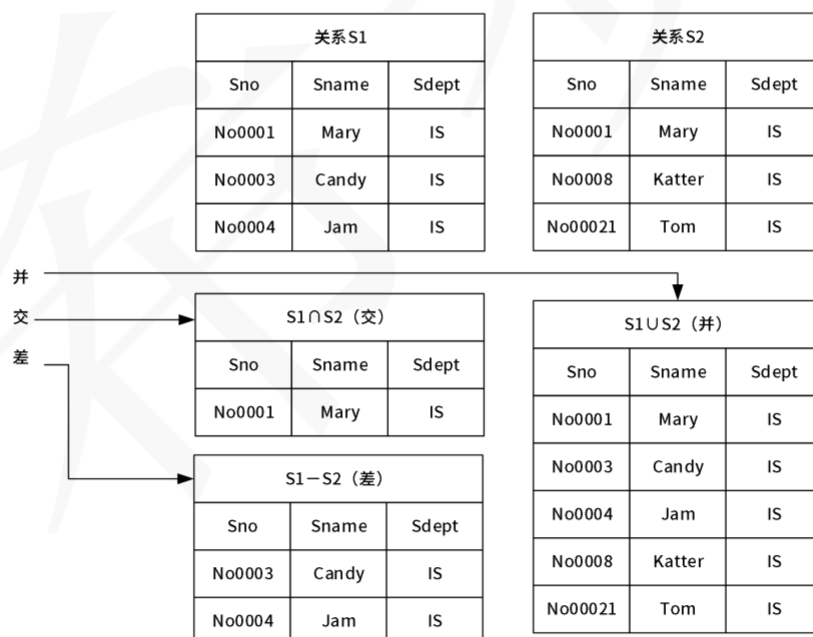
五大架构风格	子风格	核心特点及注意事项
数据流风格 Data Flow	批处理 Batch Sequential 管道-过滤器 Pipes and Filters	数据驱动, 适合用于前一步处理结果作为后一步数据处理输入的场景。
调用/返回风格 Call/Return	主程序/子程序 Main Program and Subroutine 面向对象 Object-oriented 分层架构 Layered System	层次型就属于这种, 通过分层做内容的解耦。如: 网络七层模型, B/S。

独立构件风格 Independent Components	进程通信 Communicating Processes 事件驱动系统（隐式调用） Event system	强调不直接调用，事件触发是典型。 如：订阅-发布模式。
虚拟机风格 Virtual Machine	解释器 interpreter 规则系统 Rule-based System	用于自定义场景。能不写代码自定义业务流程并执行。如：解释器。
以数据为中心 Data-centered	数据库系统 Database System 黑板系统 Blackboard System 超文本系统 Hypertext System	各种构件围绕【数据中心】工作。如：数据库系统、黑板风格。

易混淆点 13：关系代数

- 1、并（结果为二者元组之和去除重复行）
- 2、交（结果为二者重复行）
- 3、差（前者去除二者重复行）

类似于集合运算，计算如下图所示：



- 4、笛卡尔积：结果列数为二者属性列数之和，行数为二者元素数乘积。
- 5、投影：对属性列的选择列出。
- 6、选择：对元组行的选择列出。

属性名可以依次标序号，直接以数字形式出现在表达式中。计算如下图所示：

关系S1			S1 × S2 (笛卡尔积)					
Sno	Sname	Sdept	Sno	Sname	Sdept	Sno	Sname	Sdept
No0001	Mary	IS	No0001	Mary	IS	No0001	Mary	IS
No0003	Candy	IS	No0001	Mary	IS	No0008	Katter	IS
No0004	Jam	IS	No0001	Mary	IS	No00021	Tom	IS
			No0003	Candy	IS	No0001	Mary	IS
			No0003	Candy	IS	No0008	Katter	IS
			No0003	Candy	IS	No00021	Tom	IS
			No0004	Jam	IS	No0001	Mary	IS
			No0004	Jam	IS	No0008	Katter	IS
			No0004	Jam	IS	No00021	Tom	IS

关系S1		
Sno	Sname	Sdept
No0001	Mary	IS
No0003	Candy	IS
No0004	Jam	IS

$\pi_{Sno, Sname}(S1)$ (投影)	
Sno	Sname
No0001	Mary
No0003	Candy
No0004	Jam

从关系R中抽取指定的属性 (列)

$\sigma_{Sno='No0003'}(S1)$ (选择)		
Sno	Sname	Sdept
No0001	Mary	IS

从关系R中抽取满足给定限制条件记录 (行)

7、自然连接

结果列数为二者属性列数之和减去重复列,行数为二者同名属性列其值相同的结果元组。笛卡尔积、选择、投影的组合表示可以与自然连接等价。

普通连接的条件会写出,没有写出则表示为自然连接。计算如图所示:

关系S1		
Sno	Sname	Sdept
No0001	Mary	IS
No0003	Candy	IS
No0004	Jam	IS

关系S2	
Sno	Age
No0001	23
No0008	21
No00021	22

S1 ⋈ S2 (自然连接)			
Sno	Sname	Sdept	Age
No0001	Mary	IS	23

注: 笛卡尔积和自然连接比较像,要注意区分。自然连接会把重复的属性列去掉。而笛卡尔积是不会去掉重复的属性列。

8、除法

除法记为 $R \div S$, 它是笛卡尔积的逆运算。设关系 R 和 S 分别有 r 列和 s 列($r > s$, 且 $s \neq 0$), 且关系 S 的每个属性都是关系 R 的属性, 那么 $R \div S$ 的结果有 $(r-s)$ 个列。拿下列两个关系举例,

关系R				关系S	
A	B	C	D	c	d
a	b	c	d	e	f
a	b	e	f		
a	b	h	k		
b	d	e	f		
b	d	d	l		
c	k	c	d		
c	k	e	f		

$R(A, B, C, D)$ 和 $S(C, D)$, 则 $R(A, B, C, D) \div S(C, D) = (A, B)$ 。

接下来结果集要求与除数记录“c,d”和“e,f”拼接成的记录在原关系 R 中都能找到。符合这个条件的只有“a,b”和“c,k”。

易混淆点 14: 规范化理论

1、第一范式 (1NF): 在关系模式 R 中, 当且仅当所有域只包含原子值, 即每个属性都是不可再分的数据项, 则称关系模式 R 是第一范式。

2、第二范式 (2NF): 当且仅当关系模式 R 是第一范式 (1NF), 且每一个非主属性完全依赖候选键 (没有不完全依赖) 时, 则称关系模式 R 是第二范式。

3、第三范式 (3NF): 当且仅当关系模式 R 是第二范式 (2NF), 且 R 中没有非主属性传递依赖于候选键时, 则称关系模式 R 是第三范式。

4、BC 范式 (BCNF): 设 R 是一个关系模式, F 是它的依赖集, R 属于 BCNF 当且仅当其 F 中每个依赖的决定因素必定包含 R 的某个候选码。

5、第四范式 (4NF): 关系模式 $R(U, F)$ 满足 1NF, X, Y 是 U 的非空子集, $Z = U - X - Y$ 也非空, 若任取一组值对 (x, z) , 都可决定一组 y 值, 且此决定与 z 值无关, 就称 Y 多值依赖于 X , 记作 $X \twoheadrightarrow Y$ 。关系模式 R 满足 1NF, 若对任一多值依赖 $X \twoheadrightarrow Y$, X 必包含 R 的候选键, 称 R 满足 4NF。

IMAP 和 POP3 的区别是: POP3 协议允许电子邮件客户端下载服务器上的邮件,但是在客户端的操作(如移动邮件、标记已读等),不会反馈到服务器上,比如通过客户端收取了邮箱中的 3 封邮件并移动到其他文件夹,邮箱服务器上的这些邮件是没有同时被移动的。而 IMAP 客户端的操作都会反馈到服务器上,对邮件进行的操作,服务器上的邮件也会做相应的动作。

MIME 是多用途互联网邮件扩展标准,与邮件安全无关。MIME/S 与邮件安全相关。

易混淆点 16: CMM 和 CMMI

软件能力成熟度模型 (Capability Maturity Model for Software, CMM) 是一个概念模型,模型框架和表示是刚性的,不能随意改变,但模型的解释和实现有一定弹性。CMM 模型自 20 世纪 80 年代末推出,并于 20 世纪 90 年代广泛应用于软件过程的改进以来,极大地促进了软件生产率的提高和软件质量的提高。

CMMI(Capability Maturity Model Integration for Software, 软件能力成熟度模型集成)是在 CMM 的基础上发展而来的。CMMI 是由美国卡耐基梅隆大学软件工程研究所 (Software Engineering Institute, SEI) 组织全世界的软件过程改进和软件开发管理方面的专家历时四年而开发出来的,并在全世界推广实施的一种软件能力成熟度评估标准,主要用于指导软件开发过程的改进和进行软件开发能力的评估。

CMM 五级分别为: 初始级-可重复级-已定义级-已管理级-优化级

(1) 初始级。初始级是未加定义的随意过程,软件过程的特点是无秩序的,有时甚至是混乱的。软件过程定义几乎处于无章法和步骤可循的状态,软件产品所取得的成功往往依赖于极个别人的努力和机遇。

(2) 可重复级。可重复级是规则化和纪律化的过程,软件过程已建立了基本的项目管理过程,可用于对成本、进度和功能特性进行跟踪。对类似的应用项目,有章可循并能重复以往所取得的成功。

(3) 已定义级。已定义级是标准的和一致的过程,用于管理的和工程的软件过程均已文档化、标准化,并形成了整个软件组织的标准软件过程。全部项目均采用与实际情况相吻合的、适当修改后的标准软件过程来进行操作。

(4) 已管理级。已管理级是可预测的过程,软件过程和产品质量有详细的度量标准。软件过程和产品质量得到了定量的认识和控制。

(5) 优化级。优化级是持续改进的过程,通过对来自过程、新概念和新技术等方面的各种有用信息的定量分析,能够不断地、持续性地对过程进行改进。

CMMI 五级分别为: 初始级-已管理级-已定义级-定量管理级-优化管理级

(1) 初始级: 软件过程是无序的, 有时甚至是混乱的, 对过程几乎没有定义, 成功取决于个人努力。管理是反应式的。

(2) 已管理级: 建立了基本的项目管理过程来跟踪费用、进度和功能特性。制定了必要的过程纪律, 能重复早先类似应用项目取得的成功经验。

(3) 已定义级: 已将软件管理和工程两方面的过程文档化、标准化, 并综合成该组织的标准软件过程。所有项目均使用经批准、剪裁的标准软件过程来开发和维护软件, 软件产品的生产在整个软件过程是可见的。

(4) 定量管理级: 分析对软件过程和产品质量的详细度量数据, 对软件过程和产品都有定量的理解与控制。管理有一个做出结论的客观依据, 管理能够在定量的范围内预测性能。

(5) 优化级: 过程的量化反馈和先进的新思想、新技术促使过程持续不断改进。

注: 考试时需要先判断到底是 CMM 还是 CMMI 哦

易混淆点 17: 逆向工程

逆向工程的四个层级:

实现级: 包括程序的抽象语法树、符号表、过程的设计表示。

结构级: 包括反映程序分量之间相互依赖关系的信息, 例如调用图、结构图、程序和数据结构。

功能级: 包括反映程序段功能及程序段之间关系的信息, 例如数据和控制流模型。

领域级: 包括反映程序分量或程序诸实体与应用领域概念之间对应关系的信息, 例如实体关系模型。

与逆向工程相关的概念有重构、设计恢复、再工程和正向工程。

(1) 重构 (restructuring)。重构是指在同一抽象级别上转换系统描述形式。

(2) 设计恢复 (design recovery)。设计恢复是指借助工具从已有程序中抽象出有关数据设计、总体结构设计和过程设计等方面的信息。

(3) 逆向工程 (reverse engineering)。逆向工程是分析程序, 力图在比源代码更高抽象层次上建立程序的表示过程, 逆向工程是设计的恢复过程。

(4) 正向工程 (forward engineering)。正向工程是指不仅从现有系统中恢复设计信息, 而且使用该信息去改变或重构现有系统, 以改善其整体质量。

(5) 再工程 (re-engineering)。再工程是对现有系统的重新开发过程, 包括逆向工程、新需求的考虑过程和正向工程三个步骤。

注: 与逆向工程相关的概念在考试时, 往往是定义的考察, 建议死记硬背

制作于 25 年 12 月 适用于第 2 版教材

希赛网